Playable Game Building Blocks Technical PRD

Revision 0.3 • Product and engineering review • 10 September 2026

We will launch a curated discovery library of playable open-source game projects and the reusable capabilities inside them. Developers can describe the game they want to build, try a relevant demo, inspect its source, and hand a selected capability to their coding agent with enough evidence to assess and integrate it.

The initial product unit is a capability demonstrated inside a versioned project. A racing game can supply separate candidates for a vehicle controller, a chase camera and visual effects. The catalog must make those parts discoverable without implying that every visible feature is already an installable component.

Decision for this revision

Build discovery, source-backed decomposition and an agent plugin with preloaded skills first. Keep portable handoffs as an interoperability fallback. Begin with Three.js and React Three Fiber integrations. Index Phaser, Godot and Unity browser demos with explicit readiness limits. Treat Blender as an asset-authoring workflow, recorded separately from the runtime engine.

Document basis and limitations

This is a reconstructed revision of the prior Technical PRD, based on the accessible conversation titled Transcribe Game Platform Idea, including its summary of an 11-page specification. The original attachment was not available through the conversation reader and was not found in the local or connected Drive searches. This document is not a line-by-line redline and does not claim to preserve inaccessible text. Section 2 maps the recovered scope into this revision.

Playparts is a working name for the UI mockups only. No naming clearance is implied. All launch volumes, dates, budgets and success thresholds below are proposed planning targets. Mockup artwork and verification states are illustrative; repository facts and links are sourced in the appendix.

Primary outcome

A developer can find and evaluate a useful capability, then begin a bounded integration into an existing game without starting the research from scratch. Successful use is demonstrated by a working target build and preserved behavior, not by a click on an AI button.

Review owners

Product owns scope and user outcomes. Engineering owns reproducible source snapshots, extraction boundaries and integration checks. Curation owns listing evidence and metadata quality. Rights review owns unresolved licensing decisions. These are proposed responsibilities, not assigned personnel.

Scope changes from the earlier PRD

The recovered PRD emphasized a performant browser-game SDK, marketplace distribution and optional remix economics. This revision preserves the long-term direction while changing the first dependency: the catalog must create value before developers adopt a platform SDK or bring a player audience.

Earlier scope	Decision in this revision
Three.js and Phaser browser focus	Retain web performance priorities; first assisted integration supports Three.js and R3F. Phaser discovery ships first.
Marketplace and discovery	Move to the center. Index projects, playable demos and their capabilities.
Core SDK and game identity	Replace MVP SDK installation with repository URLs and versioned project records. Runtime SDK is deferred.
Accounts, saves and leaderboards	Keep developer login for saved collections and submissions. Player identity and game backend services are deferred.
GitHub and MCP agent architecture	Retain source linkage; ship a skills plugin and read-only catalog connector before hosted execution. Add evidence, manifests and validation.
Optional Remix and proprietary games	Retain optional remix. Private target games are supported without public source disclosure. Public reusable listings require appropriate rights.
Provenance and direct-parent royalties	Store source provenance early. Defer public graphs, economic agreements and all settlement.
Commerce, entitlements and hosting	Defer. Link to existing demos; optional isolated hosting comes after demand is proven.
Security, data model, reliability and metrics	Reframe around untrusted repository ingestion, demo isolation, accurate metadata and verified reuse.

Explicit MVP exclusions

No game-generation IDE, player network, store checkout, asset payment system, automatic royalty collection, generalized cross-engine conversion or automatic merging. Do not require accounts for search, demo launch or source browsing. Do not rehost third-party games as the default ingestion path.

Product wedge and later business

The catalog is useful at cold start through curation; the plugin brings its knowledge and reuse workflow into the workspace where developers already build. Future paid services may include private collections, team administration, managed verification and integration execution. Open-source rights are not contingent on buying these services. Validate willingness to pay only after repeat reuse is observed.

Users and core journeys

User	Job to complete	Evidence of value
Independent web-game developer	Find a camera, world or mechanic compatible with an existing project.	A chosen capability works in the target and preserves named systems.
Developer exploring a new game	Browse playable examples by style and mechanics.	A saved, understood starting point with an actionable source map.
Maintainer or contributor	Make an existing demo useful to other builders.	Correct listing and source credit; maintainers can correct the decomposition.
Curator	Turn public projects into trustworthy listings.	Demo/source pairing, rights status and each capability claim have evidence.

Discovery to reuse

A developer searches “low-poly racing with a chase camera.” Search separates genre, camera intent and visual style; it does not silently infer an engine. Results include a complete project and relevant capabilities, grouped to avoid duplicate flooding. The developer launches the real demo and opens the building-block tour. Selecting the camera reveals source evidence, dependencies, excluded systems and current readiness.

Use in my game asks for a target runtime and the desired change, including what to preserve. In the MVP the primary action sends the selected capability to the Playparts plugin in the user’s coding agent. The plugin loads the appropriate skills and a versioned manifest. A downloadable brief is available when a client connection is unavailable. The packet requests a pinned source revision, target inspection, a bounded patch and honest validation results. No catalog-side target write is required.

World discovery

“An explorable city based on real geography” expands into geospatial ingestion, coordinates, world streaming and movement. Suggestions such as traffic or missions are labeled additional requirements, not capabilities inferred to exist in every city demo. An attractive demo without a confirmed reusable source remains a research candidate outside the eligible open-source catalog.

State vocabulary

Keep independent labels for demo health, evidence status, rights and integration readiness. Suggested, source-reviewed and integration-tested are different states. A running demo does not prove the source snapshot matches it. A source-reviewed block does not imply a successful target integration.

Mockup mapping

Screens 1-5 cover discover, search, tour, capability and plugin handoff. Screen 6 previews later connected integration review. Screen 7 shows contributor onboarding. Screen 8 previews optional provenance. Screen 9 covers plugin onboarding and its preloaded skills. A tenth image shows mobile discovery. The HTML file is a clickable design prototype, not a functioning catalog service.

MVP requirements and release criteria

ID	Required behavior	Acceptance condition
P0-01	Catalog projects with demo and source.	Every published eligible listing has canonical URLs, a source revision, code rights, creator credit and dated demo status.
P0-02	Index project capabilities separately.	Each published capability points to source evidence at its revision and names its parent project.
P0-03	Search keywords, intent and facets.	Hard filters are honored; results explain matches; empty queries and unsupported combinations are handled.
P0-04	Launch demos with isolation.	A user gesture launches one demo; blocked embeds offer external launch; broken links show the last check.
P0-05	Show reuse and license boundaries.	The UI distinguishes source-reviewed candidates from tested integrations and code rights from asset rights.
P0-06	Export an agent handoff.	The packet includes immutable source references, evidence, requested behavior, constraints, rights and validation steps.
P0-07	Support curator and owner edits.	Changes enter a versioned review queue; an ownership claim cannot overwrite history or bypass review.
P0-09	Ship the agent plugin and preloaded skills.	A fresh supported client loads all six skills, reads the catalog, detects target context, and completes a bounded fixture integration.
P0-08	Track catalog health and outcomes.	Search, demo launch, source visit and handoff export are measurable without collecting private code.

Initial catalog target

Aim for 100 distinct projects with 2-5 reviewed capabilities each, spanning movement, vehicles, worlds, procedural generation, rendering, combat, UI, audio, multiplayer and complete starters. Treat these as launch targets, not incentives to publish thin or unlicensed entries. Start internal testing with 20 projects; expand only if review effort remains practical.

Minimum useful release

The public beta may launch with fewer than 100 projects if search evaluation and metadata gates pass. Require at least 30 source-reviewed capabilities and 10 independently reproducible reference integrations across a documented Three.js/R3F compatibility set. The plugin can support additional source-reviewed candidates, visibly marked untested. The skills package and its read-only catalog integration are required MVP deliverables, not an optional later add-on.

Complete starters and components

A project can be used whole or decomposed. Do not duplicate its code into a new package merely to manufacture a component listing. A reusable candidate may be a source slice with an integration recipe; an isolated package is a stronger readiness state earned through verification.

Search ontology and taxonomy

Use stable concept IDs with display labels, aliases, descriptions and parent relationships. Curators own a versioned ontology. AI can propose tags, but published tags must have evidence or be explicitly labeled editorial descriptions. User queries remain visible and editable after interpretation.

Facet	Representative values	Modeling rule
Content kind	Game, starter, mechanic, system, world, shader, asset workflow	One primary kind; optional secondary roles.
Genre and intent	Racing, platformer, RPG, simulation, exploration	Separate genre from capabilities.
Capabilities	Chase camera, vehicle physics, inventory, terrain, pathfinding	Hierarchical IDs; support related and requires edges.
Presentation	2D, 2.5D, 3D; first person, third person, isometric	Separate dimension, perspective and camera behavior.
Visual style	Low poly, pixel art, stylized, realistic	Editorial visual tags; do not treat as runtime facts.
Runtime stack	Three.js, R3F, Phaser, Godot, Unity; JS, TS, C#	Record versions, render API and physics libraries independently.
Input and platform	Keyboard, mouse, touch, gamepad; desktop web, mobile web, native	Record observed and declared support separately.
Assets and tools	Blender, glTF, textures, rigs, audio, procedural geometry	Blender is an authoring tool, not a runtime substitute.
World and geography	Urban, terrain, OSM, coordinates, projection, streaming	Record data-provider terms and credentials separately from code.
Reuse conditions	License, package/source slice, coupling, target support, freshness	Filter only on known values; unknown is explicit.

Query interpretation

“Mario Kart-like drifting” should map to arcade racing and drift mechanics as a search intent. It does not grant rights to protected characters, branding or assets. “Flying” expands to flight control, gliding and aerial movement, while “vehicle physics” stays distinct from a visual vehicle model. Support exact titles, repository names, spelling variants and aliases.

Ontology governance

Store concept_id, ontology_version, parent_id, aliases and deprecated_by. Keep an append-only change log. Migrations remap IDs in a background reindex without rewriting historical evidence. Require curator approval for merges that could change a hard filter. Keep user-created collection labels separate from canonical taxonomy.

Project and demo data model

A Project is a logical repository or subproject. A ProjectVersion is an immutable source snapshot. DemoBuild records how a running demo relates to that snapshot. A ComponentVersion belongs to one ProjectVersion; a reusable Component identity can accumulate reviewed versions. Never key identity solely by mutable repository URLs.

Entity	Required fields and constraints
Project	UUID; provider repository ID; canonical owner/name and URL; subproject_path; title; summary; primary content kind; creator attribution; moderation status; created_at. Unique provider ID plus subproject path.
ProjectVersion	UUID; project_id; full commit SHA; default branch at ingestion; tree digest; manifest and lockfile digests; snapshot_at; declared and observed stack; ingestion job ID. Unique project_id plus commit plus subproject path.
DemoBuild	UUID; project_version_id nullable; canonical demo URL; launch mode; claimed_revision; relation_status: proven, maintainer_claimed, unknown; browser/platform requirements; declared controls. Unknown relation cannot become tested evidence.
DemoCheck	demo_id; timestamp; final URL; reachability; interactive status; browser/device; check duration; evidence URI; error category. HTTP 200 and interactive success are separate fields.
AssetRecord	source version; path or external URL; content digest; type; format; author; license assertion; required notices; source files available; external service or dataset terms.
Listing	public slug; project/component reference; curation owner; reviewed_at; published_at; visibility; quality flags; search document version; thumbnail source and permission.

Version and relationship rules

Redirect renamed repositories using the provider repository ID. Deduplicate monorepo subprojects by path. A demo may lag the latest commit; display the relationship honestly. A moving URL is never an immutable attestation. Record the tested deployment hash if available; otherwise keep a time-stamped observation with uncertainty.

Metadata provenance

Each claim carries origin (maintainer, parser, AI proposal, curator or executed test), evidence reference, confidence, observed_at and reviewer. Preserve conflicting assertions instead of silently replacing them. Confidence measures extraction certainty, not probability of commercial compatibility. Private target metadata lives in a separate tenant boundary and is excluded from public search.

Component and integration data model

Entity	Required fields and constraints
Component	UUID; parent project_id; stable capability key; display name; purpose; canonical concept IDs; primary reuse form. A project can expose many components without duplicating the repository.
ComponentVersion	component_id; source project_version_id; source paths and symbols; entrypoints; required files; optional files; excluded systems; dependency graph; runtime ranges; coupling notes; evidence; readiness; review date. Immutable after publication.
DependencyEdge	from/to version IDs or external package; kind: imports, requires, asset, build, service; version constraint; resolved version; optional flag. Retain transitive dependencies and cycles.
LicenseAssertion	scope: project, file, asset, dataset; SPDX expression or LicenseRef; evidence path and digest; declared/concluded distinction; obligations; reviewer; resolution state.
IntegrationRecipe	component_version_id; supported target matrix; adaptation steps; configuration; tests; preserved behaviors; rollback; known limitations; recipe revision.
IntegrationRun	tenant_id; component_version_id; target source ref; request and constraints; plan; status; patch URI; result ref; test evidence; notices; resource use; actor; timestamps. MVP external reports remain unverified.
ProvenanceEdge	source_version; target_version; relationship: forked, adapted, copied, depends_on; file mappings; notice bundle; run ID; visibility. No royalty percentage inferred from the graph.

Readiness gates

Suggested means a feature may exist and stays in the curator queue. Source-reviewed means files, dependencies and applicable rights have been inspected. Isolated means the slice builds in a reference harness. Integration-tested means a named recipe passed checks against specific target versions. These gates do not replace separate demo-health and rights fields.

Database enforcement

Use foreign keys and immutable version IDs throughout. Publishing a source-reviewed component requires at least one valid evidence reference and a resolved reuse decision for the selected scope. Integration-tested requires a passing attestation with matching source, target and recipe digests. Block evidence paths that escape the source snapshot. Deleting a public listing tombstones it while retaining necessary audit and notice records.

Machine-readable contract

The downloadable schema defines a v0.1 catalog record and a sample fixture. It is an initial API contract, not a complete database migration. A separate agent handoff includes user constraints and validation requirements. Production handoffs reject unresolved commits and unsupported readiness claims; the UI prototype deliberately labels its sample export as unverified.

Ingestion and source analysis pipeline

Use a queue with independent, idempotent stages: discover -> normalize -> snapshot -> inspect -> propose capabilities -> review rights and evidence -> verify demo -> publish -> refresh. A failure in one project must not stop the catalog. Store stage outputs by source digest and analyzer version.

Source and demo intake

Accept maintainer submissions, curator-entered URLs and selected upstream catalogs. Normalize provider IDs and subproject paths; reject unsupported schemes. Fetch repository metadata and manifests with conditional requests and rate-limit backoff. Snapshot an immutable commit, then inspect source without executing it. Archive status and repository activity inform maintenance context, not automatic exclusion.

Static analysis before AI

Parse manifests, lockfiles, source imports, entrypoints, license files and asset references. For JS/TS, build an AST-based dependency graph and detect runtime and physics libraries. Record binary and generated assets without passing large binaries to the language model. Use source ranges and symbols as evidence; cap repository size, archive expansion, file count and analysis tokens.

AI decomposition

Give the model bounded source slices plus a schema and the canonical ontology. Require each proposed capability to cite a path, symbol or source range at the pinned revision. Compare import relationships against parsed facts. Reject nonexistent files, invented exports and unsupported tags. A visible effect alone is insufficient evidence that the implementation can be separated.

Review and publication

A curator confirms what the demo demonstrates, the code-to-demo relationship, source scope, creator credit and rights status. AI proposals remain private until reviewed. Moderation can reject spam, harmful embeds or misleading screenshots. Publish the project first if eligible; withhold an uncertain capability without discarding the useful project.

Refresh and failure handling

• Run lightweight demo reachability checks daily; interactive checks weekly for featured entries and after deployment changes. Back off rate-limited hosts.

• Refresh repository metadata weekly and snapshot changes on webhooks or scheduled scans. Reuse unchanged file analysis by digest.

• When relevant files or license evidence change, publish a new candidate version and mark the older tested version with its date. Never transfer tested status automatically.

• Use bounded retries with jitter, a dead-letter queue, per-host budgets and manual retry. Distinguish broken URL, denied embed, missing source, license ambiguity and analyzer failure.

AI assisted extraction and verification

Extraction produces a documented reuse boundary. It may reference existing source, create an adapter or package a slice. Prefer the smallest boundary that satisfies the requested behavior. Do not claim universal portability or silently remove source credits.

Extraction procedure

• Identify entrypoints, update-loop hooks, scene access, inputs, units, coordinates and cleanup behavior.

• Resolve direct and transitive dependencies, including shaders, textures, fonts, audio, build plugins, worker code, remote endpoints and secrets expected by the demo.

• Separate required code from optional presentation and explicitly excluded systems. Document shared state, engine assumptions and coupling.

• Inspect licenses for the selected files and dependencies. Exclude assets without adequate rights or stop for review when they are required.

• Build a minimal harness in a disposable environment when execution is allowed. Capture actual build logs, runtime behavior and resource disposal checks.

• Publish the source map, dependency manifest, adaptation recipe and verification evidence under a versioned digest. A reviewer accepts the extraction boundary.

Example boundary

For a chase camera, the candidate boundary includes follow behavior, target binding, coordinate assumptions and lifecycle cleanup. It should not pull in an entire racing UI or a second vehicle physics world merely because both are imported by the original scene. If camera logic is tightly coupled to vehicle state, record the adapter interface and the cost of separation.

Tests that establish reuse

A reference harness should exercise start, update, pause, reset and dispose; check that no listeners or render loops remain after teardown. Run the same scenario across the declared compatibility matrix. For a procedural generator, verify deterministic output for selected seeds and structural invariants such as connectivity. For a controller, validate input, collision assumptions and interaction with the target physics engine.

Failure is a valid outcome

If the system cannot establish a stable boundary, keep the entry as a source-reviewed reference with an explanation. If a test fails, retain evidence and the failing condition; do not mark it ready after a successful build alone. Record model, prompt-template, analyzer and harness versions so extraction regressions can be reproduced.

Evaluation set

Create a held-out set of 30 capabilities spanning at least six categories, including deliberately entangled examples and mixed-license assets. Two reviewers assess file-scope precision, dependency recall and false-ready claims. Proposed beta gates are >=90% supported capability claims, >=95% required dependency recall and zero known false integration-tested labels.

Agent plugin and preloaded skills

The plugin is a core MVP product surface. Developers should be able to search the catalog and use a building block from inside their coding agent, with the right game-development guidance already available. Installing a plugin is not equivalent to authorizing all future source changes.

Included skills

Skill	Purpose and output
Find building blocks	Interpret game intent, search the catalog, compare playable candidates and report stack and rights limits.
Inspect source and target	Map source evidence and inspect local engine, versions, physics, render loop, asset pipeline and existing project instructions.
Plan an integration	Define a bounded capability slice, required dependencies, systems to preserve, rights evidence and a target-specific plan.
Integrate a building block	Apply an authorized same-stack adaptation in an isolated workspace using the host agent's existing file tools. Stop on missing required rights or contradictory target constraints.
Validate the result	Run meaningful checks, record source/target/recipe versions and report actual results, failures and untested conditions.
Preserve credits and provenance	Create required notices and local source mappings; keep optional public remix publishing as a separate choice.

Preloaded knowledge and project context

Bundle the ontology definitions, readiness vocabulary, catalog/handoff schemas, stack-specific adapter guidance, validation checklists and provenance format. Load only the selected skill and relevant references. Inspect the target once per source state; cache a non-sensitive context summary keyed by target manifest digest. Record engine versions, physics library, coordinate units, render-loop ownership, asset formats and user constraints. Never silently change an existing project instruction file.

Primary user journey

Use in my game passes a component version and intent into the installed plugin. The plugin resolves the versioned source, inspects the target, creates a plan, performs authorized local edits and runs validation. The catalog is the knowledge service; the existing agent provides reasoning and workspace tools. Users can also start in the agent with "Find a controller for this game." Portable export remains available for unsupported clients.

Support scope

First packaging target: a Codex-compatible plugin with Markdown skills, selected because it can be delivered as a concrete starter here. Keep the domain contracts provider-neutral. Other agent-client adapters require separate installation, permissions and handoff tests. Three.js/R3F is the first integration family; Unity and Godot skills initially guide discovery and feasibility only.

Plugin packaging permissions and release gates

Version the plugin independently from the catalog API, ontology and individual recipes. The manifest declares actual packaged capabilities only. A released plugin contains skills, reference resources and a real catalog connector configuration; it must not advertise an endpoint or executable that does not exist.

Layer	Implementation requirement
Skills package	Six focused skills, progressively loaded references, manifest, semantic version and compatibility metadata. Signed release artifacts and release notes are a production requirement.
Catalog connection	Read-only search, component details, evidence and handoff tools. Public browsing needs no private-repository permission. Authenticate only for saved or private catalog resources.
Host workspace	The coding agent’s existing workspace tools perform inspection and authorized edits. Restrict writes to the selected target and preserve its instructions.
Optional hosted execution	Later phase. Separate repository-scoped connection, worker credentials, plans, cancellation and reviewable patches.
Local context	Runtime/manifest summary and preserve constraints. Keep secrets out of the catalog, telemetry, exported evidence and model-visible logs.
Update and rollback	Pin plugin, schema and recipe versions in each run. Detect incompatible contracts; offer an explicit compatible update or fallback. Preserve the previous package for rollback.

Onboarding and permissions

The plugin setup screen shows the included skills, supported clients and connection state. Choose a workspace only when using a block. Explain public catalog reads separately from local file inspection and code changes. A successful install acknowledgment, successful catalog call and compatible manifest response form the connection check. If the catalog is offline, bundled examples remain labeled research candidates; no stale record may retain a fabricated fresh verification date.

MVP acceptance tests

• In a clean supported client, discover all six skills and successfully retrieve a catalog component and its evidence.

• From both website and agent entry points, preserve the selected component version and user constraints through planning and integration.

• Integrate a reviewed Three.js/R3F fixture, validate it and include correct notices; reject missing rights, stale target state and incompatible engine assumptions.

• Handle offline catalog, expired authentication, denied local access, unsupported schema, interrupted execution and plugin rollback with actionable messages.

• Track activation through a successful catalog lookup and first reviewed plan. Measure weekly plugin-assisted verified reuse separately from exports and self-reports.

Starter package delivered with this PRD

The downloadable starter contains real skill instructions, bundled references and a local context-inspection helper. Its manifest is validated. It is not installed in this environment and includes no live catalog server, production signing or hosted executor. Those are explicit implementation items for the released MVP. The starter helps review and begin the plugin work without presenting a scaffold as a finished connected product.

Agent handoff and target integration

The MVP ships a downloadable agent plugin with preloaded skills, a read-only catalog connection and a portable Markdown/JSON handoff format. It includes the immutable source revision, component version, source evidence, dependency and notice references, target runtime supplied by the user, desired behavior, preserved systems and a validation checklist. Exporting is useful without catalog access to the private target repository.

MVP interaction contract

The UI offers Use with Playparts plugin as the primary action, with a supported-client connection check and a target workspace choice. First-time users see the included skills and access requirements before setup. If no supported client is connected, offer Download starter plugin and Download agent handoff. Show Installed only after client acknowledgment; Integrated requires a validated target result. Provider-specific launch behavior must be tested rather than assumed.

Connected integration in a later phase

The user selects a specific repository and branch through a scoped connection. Read access supports target inspection; separate write authorization permits a branch or patch. The agent reads manifests and a bounded target map, compares runtime versions, detects conflicting systems and proposes a plan. A source or target change invalidates the plan until rechecked.

• Start from an isolated branch or worktree. Record target base SHA, component version and requested constraints.

• Apply a bounded patch; do not run arbitrary repository instructions as trusted agent commands. Prohibit secret access beyond explicitly scoped integration needs.

• Run existing tests and capability-specific checks. Compare behavior and resource use against the unchanged target baseline on a declared device profile.

• Return changed files, dependency changes, notices, actual test results, limitations and a preview when available.

• The user reviews the patch. Never merge, deploy, publish a remix, or accept economic terms merely because Use in my game was clicked.

Compatibility behavior

A Three.js/R3F target can be eligible only for the tested version ranges. Conflicting physics engines or render-loop ownership trigger adaptation review. Unity or Godot targets receive a reference and feasibility assessment in the initial release; no JavaScript-to-C# or engine conversion promise is made. Assets authored in Blender may be reusable through supported export formats after their own rights and format checks.

Failure and rollback

Model the run as queued, inspecting, needs_input, planned, applying, validating, review_ready, failed, cancelled or accepted. Cancellation kills workers and preserves a report. On failure, return the isolated patch and logs without changing the target base. Rollback is branch deletion or patch reversal; the acceptance flow records the final target revision and user action.

Search retrieval and ranking

Search must retrieve both project records and component records. Use Postgres full-text and trigram search for lexical recall, a vector index for semantic intent, and typed facets for exact constraints. Start in one database; add a dedicated search service only when measured scale or latency justifies it.

Candidate retrieval

Parse the query into optional intent facets with confidence. Apply explicit hard filters first, including runtime and rights/readiness. Union the top lexical and semantic candidates, then fuse ranks with reciprocal rank fusion using k=60. Group versions under stable identities and suppress near-duplicate forks unless they offer a meaningful capability difference.

Proposed initial reranking

For eligible candidates, normalize each feature to [0,1]. Use 0.45 relevance + 0.20 target compatibility + 0.15 evidence quality + 0.10 demo reliability + 0.05 freshness + 0.05 curator quality. Relevance is the normalized fused retrieval score. If no target is known, remove compatibility and renormalize the remaining weights. These are starting weights to tune against judged queries, not validated product findings.

Rights and hard runtime constraints are eligibility gates, not small scoring penalties. Unknown compatibility stays unknown and is excluded when the user asks for tested support. Source-reviewed candidates can appear in general discovery with their status. With at least ten distinct relevant projects available, show at most two results from one repository in the top ten and offer expansion for its other parts.

Explainability and quality

Explain matches using actual indexed evidence: “Matches chase camera; demonstrated in a racing game.” Show when style differs or integration is untested. GitHub stars may be shown with a timestamp as context but do not dominate ranking. Avoid fake use counts, unverifiable time-saved claims and popularity loops. Promotional placements, if introduced, are labeled separately from organic relevance.

Evaluation and feedback

Use 100 judged queries covering exact titles, plain-language concepts, rich composites, geospatial intent, stack constraints, license filters and no-result cases. Split tuning and held-out sets; use graded relevance 0-3 and report NDCG@10, Recall@20 and constraint violations. Proposed launch gates: NDCG@10 >=0.75, Recall@20 >=0.90 and zero hard-filter violations in the suite.

Log query intent, applied filters, result IDs, explanation versions and outcomes with privacy controls. Let users correct interpretation. When there are no matches, offer explicit filter relaxation or related capabilities; do not silently ignore requirements. Cache by query, facets, tenant scope and index version; never mix private and public candidates.

Architecture and execution boundaries

Use a modular web application with one metadata database, an object store for immutable evidence, a durable job queue and isolated analysis workers. Keep the public browsing path independent of slow source analysis and game execution. This architecture is a proposal, with provider choices left open until cost and operational needs are measured.

Layer	Responsibility	Boundary
Catalog web UI and API	Search, tours, source maps, collections, submissions and exports.	No third-party game scripts execute in the catalog origin.
Metadata store	Versioned projects, components, ontology, moderation and tenant records.	Foreign keys, tenant authorization and immutable snapshot IDs.
Search index	Full-text, vectors, typed facets and ranking features.	Derived data rebuilt from reviewed metadata; private index scope enforced.
Object storage	Evidence, manifests, permitted thumbnails, logs and patches.	Content digests; signed access for private artifacts; retention policies.
Ingestion workers	Fetch, parse, propose and refresh.	No target credentials; no execution during static inspection.
Verification workers	Build reference harnesses and later target integrations.	Disposable sandbox; CPU, memory, network and time limits.
Connection broker	Later-phase GitHub installation and short-lived credentials.	Scoped repository access; secrets unavailable to public demo origin.

Data flow

A submission creates an ingestion job. Its immutable analysis artifacts feed a curator review. Publishing commits metadata and an outbox event atomically; an index worker consumes the event idempotently. Search returns reviewed listing IDs. The handoff service resolves a selected component version and emits a digest-bound packet. Later integration jobs read that same packet and scoped target metadata.

Demo isolation

Prefer external launch. If embedding is supported, host the demo on a separate origin with a restrictive sandbox and only necessary permissions. Do not combine scripts and same-origin privileges for untrusted same-origin content. Pointer lock, fullscreen and audio require user interaction and a visible exit. Validate any postMessage source and origin. These browser constraints follow the iframe model documented by MDN [9].

Operational recovery

Use transactional outbox delivery, retry-safe job IDs and source-digest caches. Keep the previous working public index during rebuilds. Back up the metadata store daily with point-in-time recovery where supported; proposed RPO is 24 hours and RTO is 4 hours for beta. Test restoration before launch. Indexes are rebuildable; source evidence and review decisions are durable.

APIs and security requirements

Proposed endpoint	Contract
GET /v1/search	q, type, facets, target_profile and cursor. Returns items, match reasons, readiness, applied filters and stable next_cursor.
GET /v1/projects/{id}	Current public listing plus version references, demo health and attribution.
GET /v1/components/{id}/versions/{version}	Immutable source scope, evidence, dependencies, rights and recipes.
POST /v1/submissions	Repo URL, subproject path, demo URL and optional ownership claim. Returns 202 with job_id and status URL.
POST /v1/handoffs	component_version_id, target profile, intent, constraints. Returns manifest and notice references; unresolved required evidence returns 422.
POST /v1/integrations	Later: scoped target connection, base SHA, handoff digest and approved plan. Requires authorization and idempotency key.
GET /v1/jobs/{id}	Scoped stage, progress, result references and actionable failure code.
POST /v1/reports	Listing issue, category and evidence. Rate-limited; does not publish the reporter’s private information.

Protocol behavior

Use opaque cursor pagination with stable tie-breakers. Version schemas and enforce request validation. Return 401 for missing authentication, 403 for scope denial, 409 for a stale target or plan, 422 for unsupported inputs or unresolved rights, and 429 with Retry-After for limits. Errors use code, message, retriable and action fields. Retryable mutations use tenant-scoped idempotency keys.

Agent integration surface

The MVP plugin’s read-only catalog MCP server exposes search_components, get_component and prepare_handoff as scoped tools backed by the same API. Mark retrieval tools read-only and keep execution separate. Tool descriptions must accurately state side effects; follow the chosen MCP protocol version and require user control for consequential calls [10]. Pin and test protocol compatibility before release.

Security controls

• Treat repository files, README text and model output as untrusted data. Deny instruction escalation through retrieved content. Validate model JSON and evidence paths.

• Defend fetchers against SSRF, private IPs, metadata endpoints, redirect rebinding, archive traversal, decompression bombs and oversized binaries.

• Build untrusted code in ephemeral workers with no platform secrets, bounded resources and default-denied network access. Allow only explicitly reviewed package and artifact fetches.

• Use least-privilege connections, encrypted secrets, short-lived worker credentials and signed webhook verification. Audit reads and writes to private targets.

• Proposed retention: delete private working copies within 24 hours after a run and private logs after 30 days unless users retain them. Scrub secrets, honor deletion requests and exclude private code from model training or public indexing.

Licensing attribution and provenance

Public repository visibility is not a license grant. GitHub documents that normal copyright applies when no license is provided [6]. An open-source listing must have a resolved code license appropriate to the represented source. Separate reuse eligibility for selected files, assets, third-party dependencies and datasets.

Rights decision rules

Finding	Catalog and reuse behavior
Recognized permissive code license	Record SPDX expression, exact evidence and notice requirements. Selected files and assets still need scope review.
Copyleft or mixed license terms	May be discoverable with accurate terms. Route integration compatibility and distribution obligations for review; do not label universally safe for commercial use.
No license or contradictory evidence	Keep as a research candidate outside the eligible open-source catalog; disable extraction/export of source content. Link-only research context must state the uncertainty.
Third-party assets or map data	Track each rights source, attribution, redistribution conditions and service dependency. A code license does not clear an asset or dataset.
Blender source files	Record .blend availability and export formats. Inspect the asset's own license; do not infer it from the authoring tool.
Optional commercial agreement	Store separately from the software license, with explicit acceptance. No default royalty or settlement in MVP.

Attribution bundle

Every export includes source project and creator references, selected commit, applicable license texts or durable references, notices, asset credits and a record of modifications. Store declared and reviewed license conclusions separately. SPDX identifiers and expressions standardize how license information is represented [7]; they do not decide whether a particular integration satisfies its obligations.

Provenance model

Record forked_from for whole-project derivatives and adapted_from or copied_from for component composition. Preserve original and immediate source relationships so multiple contributors can receive correct credit. Dependency edges are not proof of authorship. Keep private target identifiers out of the public graph unless the owner chooses to publish them.

Provenance recording is optional as a platform feature; applicable license notices remain required. A later public remix publishes its chosen source and lineage visibility only after a separate action. Royalty obligations are not inferred from open-source status or graph edges. The Open Source Definition does not allow a royalty or fee requirement for redistribution [8]. Any optional network economics need their own agreement and implementation phase.

Corrections and removal

Provide maintainer corrections, rights reports and takedown review. Freeze new exports for the affected scope during a credible unresolved rights issue. Preserve an audit record and notify affected users where appropriate; do not quietly rewrite historical source attribution.

Developer onboarding and catalog operations

Contributor onboarding

A developer enters a repository URL, playable demo URL, runtime and a brief description of useful capabilities. Optional fields cover subproject path, source-to-demo mapping, asset-authoring tools and a project manifest. No SDK installation or platform hosting is required. Anonymous suggestions may enter a rate-limited queue; edits to claimed listings require verified control.

A GitHub ownership check can verify maintainer access for the specified repository. Ownership of the repository is not proof of rights to every included asset. Show a draft listing with proposed capabilities and evidence. The developer can correct descriptions, exclusions and notices; curation confirms publication. AI-generated metadata is identified in the review history.

Developer consuming a block

Browsing and external demo launch work without login. Save collections locally initially, with optional account sync. Ask only for target details that affect compatibility. The plugin runs inside the user’s existing coding agent and keeps target inspection and authorized edits local in the MVP. Later repository connection requests explain selected repository access and separate inspection from writing changes.

Review queue and health

Curators see actionable queues for missing licenses, uncertain demo/source pairs, invalid evidence, broken demos and unreviewed updates. Each queue item identifies the blocking field and next step. A flagged demo can remain visible with a broken-state explanation when the source remains useful; it must not carry a playable-now badge.

Metadata responsibility	Refresh and quality rule
Title, purpose, capability map	Reviewed on initial publication and relevant source changes.
Engine and dependency versions	Parsed per source snapshot; compare with maintainer declarations.
Demo status and controls	Date each observation; test keyboard and touch claims separately.
Screenshots and preview media	Capture only with suitable rights; record source, date and whether illustrative.
License and notice evidence	Digest at source version; re-review changed scope before new exports.
Stars, forks and activity	Optional time-stamped context; never a substitute for functional verification.

Service and accessibility

Proposed curation target is a first response within three business days during beta. Let maintainers appeal errors. Provide keyboard-operable navigation, visible focus, clear external-link behavior, text descriptions for previews and accessible filter labels. Mobile discovery should remain useful even when a desktop-only demo cannot launch; offer save-for-desktop and source browsing.

Performance metrics and validation

The catalog shell has a separate performance budget from external games. Load preview images first and one demo only after a user gesture. Unload or pause it on exit. Measure shell and game behavior independently; never advertise a catalog-wide frame-rate guarantee based on a few demos.

Measure	Proposed beta target and measurement
Search latency	Server p95 <=700 ms at 20 requests/second over a 10,000-component test index; record region and warm/cold mix.
Page experience	p75 LCP <=2.5 s, INP <=200 ms and CLS <=0.1 for the catalog on a declared mid-range mobile test profile.
Initial payload	<=250 KB compressed application JavaScript excluding third-party demos; defer game code and heavy preview media.
Availability	99.5% monthly for the catalog API; external demo availability reported separately.
Handoff generation	p95 <=3 s for export of already-reviewed metadata; extraction and model execution are asynchronous.
Reference integration	Build and functional checks pass on the declared matrix; frame time, memory and loading deltas reported against baseline.
Public data completeness	100% of eligible published records satisfy required demo, source, revision, license and credit fields.

Product outcome metrics

North-star metric: weekly developers with at least one verified successful reuse, counted once per developer per week. Verification requires a run attestation with matching source/target revisions and passing defined checks, or independently reviewed external evidence. MVP handoff exports are a leading indicator, not this north-star outcome.

Measure search-to-qualified-detail rate, detail-to-demo-launch, source-open rate, handoff export, user-reported success and repeat use at four weeks. Distinguish self-reported outcomes from verified outcomes. Proposed pilot gates: 20 external developers, at least 10 verified integrations and at least five developers returning to attempt a second reuse within four weeks. These are decision thresholds, not forecasts.

Validation gates

• Run the held-out search suite, explicit runtime/license filters and zero-result recovery before release.

• Verify a demo blocked by frame policy, a broken demo, an unknown source revision, a missing license and a removed repository.

• Test extraction with malicious README instructions, traversal paths, missing transitive assets and conflicting package versions.

• Test handoff reproducibility, notices and preserved-behavior constraints. Verify target changes invalidate later connected plans.

• Run tenant-isolation and deletion tests before accepting private target data. Exercise backup restore and queue replay.

Roadmap dependencies and open decisions

Use evidence gates instead of fixed launch dates. The illustrative sequence below assumes a small team covering product/design, web engineering, analysis infrastructure and curation. Effort estimates must be revisited after the first ten source reviews reveal actual extraction cost.

Phase	Deliverables	Exit gate
0 Curation prototype	20 projects, ontology v0.1, metadata schema, rights checklist and mockup interviews.	Five target developers can find a useful part; demo/source review workflow works.
1 Public discovery MVP	Reviewed catalog, hybrid search, tours, skills plugin, read-only catalog MCP, local-agent workflow and handoff fallback.	MVP acceptance criteria and search quality gates pass; reference integrations reproduced.
2 Assisted verification	Extraction harnesses, package/source boundaries, held-out evaluation and signed result records.	False-ready and dependency-recall gates pass at an affordable review cost.
3 Connected integrations	Scoped target connections, plans, patches, validation, cancellation and rollback.	Pilot reuse outcomes and security checks pass; no automatic merge or publication.
4 Broader engines and remixes	Phaser recipes, Godot/Unity adapters, Blender asset pipelines and optional public provenance.	Each engine earns its own tested matrix; rights and format boundaries are documented.
5 Platform services	Potential private teams, managed hosting, runtime services and optional commerce.	Repeat demand supports a business case; economics reviewed separately.

Risks and product responses

Curation may cost more than indexing. Track reviewer minutes per accepted capability and favor dense, well-documented projects. Extracted code may be brittle: preserve specific tested versions and show limits. Demo URLs decay: retain source context and dated health. Broad engine coverage can dilute quality: expand discovery ahead of integration support. Search may surface inspiring but incompatible results: separate reference value from target compatibility.

Decisions to resolve through the pilot

• Choose the first ten target integrations and supported runtime/version matrix.

• Test plugin activation and local integration success before funding hosted target execution.

• Measure curator cost, model cost, sandbox minutes and storage per accepted capability; set budgets from observed data.

• Decide whether browser-only demos are sufficient at launch; retain native-only examples as a clearly separate future category.

• Choose the product name and initial distribution channels after interviews. Confirm whether private collections have paid demand.

Release recommendation

Approve the catalog and skills-plugin wedge for implementation. Treat broad automatic extraction and cross-engine integration as hypotheses requiring measured verification. Defer the earlier platform SDK, commerce and royalty work until repeat developer reuse is established.

Seed catalog and evidence notes

These are seed candidates checked against public primary pages on 10 September 2026. Repository pages and linked demo destinations were inspected as research sources; no claim is made that every demo was interactively played or that any extraction was completed. Capability labels below are proposed catalog decompositions until source-level review. The supplied seed-catalog.json preserves direct URLs and uncertainty.

Candidate	Proposed useful parts	Current evidence and launch treatment
pmndrs Racing Game [1]	Vehicle behavior, camera, dust/skid effects, UI; Blender-to-glTF workflow.	Repository links a live demo and describes R3F, MIT code and CC0 assets. Confirm selected files at a pinned revision.
Ecctrl [2]	Character control, custom gravity, vehicles and input.	Repository describes an R3F/Rapier controller toolkit and links its demo. MIT at repository level; per-scope review still required.
Dungeon Forge [3]	Seeded generation, connected room graph, themes and post-processing.	Repository describes a Three.js generator, MIT license and live demo. Source boundary needs review.
Godot Demo Projects [4]	2D/3D movement, rendering, physics and UI templates.	Official source collection and browser-demo catalog. Inspect license and export availability per subproject. Initial integration status is reference only.
Gather It [5]	Unity resource collection and worker behavior.	Public repository and linked browser build. No license was established from the inspected page. Research candidate; exclude from reusable open-source launch catalog until cleared.
San Francisco game [11]	Geospatial city exploration and world-system research.	Live project supplied in the earlier conversation. Reusable source and its license were not established here. Research candidate, not an eligible open-source block.

How these examples shape the product

The catalog needs a place for complete games, focused toolkits and individual demos inside larger repositories. Runtime, authoring tools and reusable capabilities cannot share a single tag field. It also needs an honest boundary between an appealing public example and a source-backed reusable listing.

Mockup evidence policy

The image-generation tool produced fictional game artwork for the UI. Those images are labeled illustrative and must not be presented as screenshots of the named repositories. Sample verification states, target project Coastal Rally, file-change counts and provenance relationships are design fixtures. External source and demo links point to actual projects; linking does not imply endorsement or successful integration.

Primary references and document history

[1] pmndrs Racing Game repository and README
https://github.com/pmndrs/racing-game

[2] Ecctrl repository and documentation
https://github.com/pmndrs/ecctrl

[3] Dungeon Forge repository and README
https://github.com/majidmanzarpour/threejs-procedural-dungeon

[4] Godot official demo projects
https://github.com/godotengine/godot-demo-projects

[5] Gather It Unity project
https://github.com/tweeres04/gather-it

[6] GitHub documentation on licensing a repository
https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/licensing-a-repository

[7] SPDX guidance on handling license information
https://spdx.dev/learn/handling-license-info/

[8] Open Source Initiative Open Source Definition
https://opensource.org/osd

[9] MDN iframe element and sandbox behavior
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/iframe

[10] MCP tools specification version 2025 06 18
https://modelcontextprotocol.io/specification/2025-06-18/server/tools

[11] San Francisco game supplied as a geospatial reference
https://sf.thijs.gg/

History

Earlier specification: an 11-page browser-game platform PRD described in the source conversation. Revision 0.3: reconstructed around the building-block discovery wedge, including a core preloaded skills plugin, scope migration map, requirements, source-backed ontology, data contracts, extraction and integration boundaries, ranking, rights handling, operations, validation and roadmap.

The public references support the cited ecosystem facts and protocol or licensing principles. Architecture choices, thresholds, entity definitions, workflows and rollout gates are proposals made in this PRD. Final implementation must check the protocol and dependency versions chosen at build time.